

UNIVERSITÉ PARIS CITÉ

M2 INTERNSHIP REPORT

Compositional reasoning for contextual preorders

Savan Kan

Supervisors:
Hugo Férée and Giovanni Bernardi

September 7, 2026

Contents

1	Introduction	2
2	Preliminaries	3
2.1	Syntax	3
2.2	Semantics	4
2.3	Convergence	6
2.4	The must preorder	7
3	Main contributions	8
3.1	The contextual characterization	10
3.2	The alternative characterization	11
3.3	Convergence of parallel composition	12
3.4	Zippping	13
4	Discussion and future works	14
5	Conclusion	15
6	Appendix: selected compositionality proofs on pen and paper	16

1 Introduction

When studying the metatheory of programming languages, we are usually interested in whether two programs t and u are equivalent which means we are interested in some-kind of program equivalence “ $t \equiv u$ ”: for instance β -equivalence or $\beta\eta$ -equivalence. But one might also ask whether u preserves the properties of t for instance when trying to show that after appending (and thus modifying the original program) new code to the program t resulting in program u ; we haven’t broken any invariants. Such preservation property can be expressed not as program equivalence (because otherwise we can’t append new code) but as program *preorders*.

We are interested in such preorders say $t \leq u$ that express preservation of properties when going from t to u . But now let’s consider a programming language with a very well known complex feature: concurrency! Concurrency is very useful for making programs run faster but the drawback is that it is very hard to handle it properly; and one reason is non-determinism.

In the concurrent setting, the *must-preorder* is the kind of preorder that we are interested in i.e. a preorder that express some kind of property preservation. The property that we wish to study is liveness, this property states that “something good eventually happens” and is something common to consider in concurrency theory. The *must-preorder* $\sqsubseteq$ expresses preservation of liveness as follows:

$$p \sqsubseteq q \iff \forall e. p \text{ MUST } e \text{ implies } q \text{ MUST } e \quad (*)$$

Where $p \text{ MUST } e$ is a predicate that expresses liveness by saying that either the test e is good now with respect to p or e will later be good with respect to p . The preorder is known to have two other equivalent (modulo some details) alternative characterizations: one which is based on convergence and resolution of deadlocks, and the other which is just the coinductive formulation of the latter.

Goal of the internship and report structure

The must preorder has a pretty simple definition (*) and yet it is quite hard to come up with some actual concrete examples of processes p and q that satisfy $p \sqsubseteq q$. However there are some known interesting properties that this preorder satisfies that can help us build concrete examples of such p and q , namely *compositionality* with respect to some constructs.

The goal of the internship was to mechanize in the Rocq proof assistant those compositionality properties as part of an ongoing work ([1], [2]) of mechanizing a particular topic of concurrency theory called *testing theory* which revolves around the must preorder.

But we also had another goal during the internship which was to compare the various characterizations of the preorder to see which one of those characterizations are easier to use for reasoning on the preorder. As previously mentioned, there are three characterization for the must-preorder: basic/inductive (*), alternative, coinductive. In this report, we will focus only at the first two characterizations.

The carried work is based on the recent papers [1] and [2] and has been mechanized/formalized within the Rocq proof assistant. The Rocq formalization that the author of this report has been working on is based upon the one of Gaëtan Lopez who is still currently working on it.

The report is organized as follows: Section 2 gives the preliminary background needed to understand the contributions of the internship, Section 3 gives the main contributions of the internship, Section 4 discusses future work, and finally Section 5 gives conclusions. We also include an appendix containing pen and paper proofs of some of the compositionality proofs.

As in recent papers on Rocq mechanizations like [2] and [5], we put hyperlinks with the Rocq logo  to codes of our Rocq developments to avoid polluting the report with too much Rocq code; in fact, no Rocq code will be written in this report !

Main contributions

We gave a proof in Rocq of compositionality for the contextual characterization $\sqsubseteq$ in the case of the following constructs: internal communications $\tau.p$, receivers $a?(x).p$, internal sums $p \oplus q$, channel restrictions $(\nu a) p$, parallel composition $p|q$; which we discuss in Section 3.1. We gave a proof in Rocq of compositionality for the alternative characterization $\preceq_{AS}$ in the case of those same constructs except for $p|q$; we discuss the proofs in Section 3.2. We formalized in Rocq a counterexample for compositionality of the preorder (for both characterization) with the sum $p + q$; which we discuss in Section 3.1 and 3.2. We formalized in Rocq the notion of *zipping* that characterizes the weak transitions of $p|q$; we discuss it in Section 3.4. We gave evidence in Section 3.3 that in order to prove the compositionality for $\preceq_{AS}$ in the case of $p|q$, the convergence part for the proof would likely need very strong convergence hypotheses if we don't appeal to acceptance sets; and we gave evidence hinting that the difficulty for ensuring convergence of $p|q$ lies in the definition of convergence on a trace and possible of interactions between p and q .

2 Preliminaries

We note that process algebras or process calculi are meant to model/represent distributed systems and concurrent communication between processes, so they are not programming languages and thus are not meant to be used for writing programs.

The work done by Bernardi and his colleagues are meant to be independent of a fixed process algebra i.e. their results in [1] and [2] are meant to be applicable to various process algebras like CCS, VCCS, VACCS, π -calculus etc. But in our work we focus for the sake of simplicity on VACCS¹ which has two central characteristics: (1) the communications are done by sending and receiving *values*, (2) the communications are *asynchronous*; the asynchrony here manifest itself specifically in the fact that senders don't have continuations (but receivers do).

The syntax of VACCS contains binders, so to reason on VACCS' processes in Rocq, we need some way to represent binders and in the Rocq code that we work on, we use De Bruijn indices to do so. We shall not talk in details about De Bruijn indices in this report as they are not the main topic of this report. For curious readers wanting to know more about how to use De Bruijn indices in process calculi; we invite them to read the paper of Kan and Lenglet [5].

2.1 Syntax

The syntax of that calculus is given in Figure 1. Now let us describe the syntax in details.

We use the letters a, b, c, d to range over channel names; and x, y, z to range over value variables; and X, Y, Z to range over process variables. The letter v will range over values.

- The process $p|q$ is the process that runs the two process p and q in parallel.
- The process $a!v$ is called a *sender* (also called *output*), what it does is send along the channel a a value v .
- The process $(\nu a) p$ *restricts* the visibility of the channel a to solely p i.e. it makes the channel a private and only accessible inside p . It binds the name a in p .
- The process *if* b *then* p *else* q evaluates the boolean expression b then branches to either p' or q' depending on the result of the evaluation of b , where p' and q' are respectively some transitions of p and q . This ifthenelse construct is not like the regular one in a programming language: in the semantics of the ifthenelse construct of VACCS, we do two reduction steps at once².
- The process X is a *process variable* used solely for the recursion process.
- The process $\text{rec}X.p$ is the *recursion process*, it evolves by first unfolding itself to $p[(\text{rec}X.p)/X]$. It binds X in p .

processes	p, q, r, e, P, Q, R	$::=$	$p q$ $a!w$ $(\nu a) p$ $\text{if } b \text{ then } p \text{ else } q$ X $\text{rec}X.p$ g	parallel composition send restriction branching process variable recursion guard
guards	g	$::=$	$\mathbf{0}$ $a?(x).p$ $\tau.p$ $g_1 + g_2$ $\mathbf{1}$	inaction receive internal communication choice success
messages	w	$::=$	$v \mid x$	

$$p \oplus q \stackrel{\text{def}}{=} (\tau.p) + (\tau.q)$$

$$\Omega \stackrel{\text{def}}{=} \text{rec}X.X$$

Figure 1: Syntax of VACCS and derived forms

- the process $\mathbf{0}$ is the process that does nothing.
- the process $a?(x).p$ is called a *receiver* (also called *input*), what it does is waiting to receive on channel a a value which it then binds to x and continue on by executing p . The process binds x in p .
- the process $\tau.p$ does some internal action that is unknown to us (depicted with a symbol τ) then continues on by executing p .
- the process $g_1 + g_2$ evolves by either evolving into g_1 or g_2 and continues execution along one of the *non-deterministically* chosen g_i (i.e. the other one is forgotten).
- The process $\mathbf{1}$ is here to denote a *successful test*, it solely in the syntax so that it can be used to define the predicate MUST (we will present it later in the report).

From those listed basic constructs, we can define two special constructs: (1) the internal sum $p \oplus q$ which reduces either into p or into q , (2) the process Ω that reduces into itself and is of course an infinitely reducing process.

2.2 Semantics

The operational semantics for our calculus VACCS is given by labeled transitions, it is therefore referred to as an LTS (labeled transition system); it is given in Figure 2. We note that there are two types of actions: visible actions (denoted by the *metavariable* μ) and internal/invisible actions (denoted by the *symbol* τ); in other words an action is either τ or a μ .

The rule RESMU has the side condition ³ $\text{chan}(\mu) \neq a$ which is here to ensure that we can do a visible action $(\nu a) p \xrightarrow{\mu} _$ as long as we are not doing it on the restrained channel a : that's the whole purpose of the restriction operator ν .

²VACCS = Value passing Asynchronous Calculus of Communicating Systems

²In a regular programming language, if $\text{true then } p \text{ else } q$ would reduce in one step to p ; but in VACCS, that expression doesn't necessarily reduce to p but reduces to p' a reduction of p .

³ $\text{chan}(\mu)$ is defined as follows: $\text{chan}(a?v) = \text{chan}(a!v) = a$

$$\begin{array}{l}
\text{actions} \quad \alpha ::= \mu \mid \tau \\
\text{visible actions} \quad \mu ::= a!v \mid a?v
\end{array}$$

$$\begin{array}{c}
\text{RCV} \quad \frac{}{a?(x).p \xrightarrow{a?v} p[v/x]} \quad \text{SEND} \quad \frac{}{a!v \xrightarrow{a!v} \mathbf{0}} \quad \text{TAU} \quad \frac{}{\tau.p \xrightarrow{\tau} p} \quad \text{REC} \quad \frac{}{\text{rec } X.p \xrightarrow{\tau} p[(\text{rec } X.p)/X]} \\
\\
\text{IFTRUE} \quad \frac{p \xrightarrow{\alpha} p' \quad \llbracket b \rrbracket = \text{true}}{\text{if } b \text{ then } p \text{ else } q \xrightarrow{\alpha} p'} \quad \text{IFFALSE} \quad \frac{q \xrightarrow{\alpha} q' \quad \llbracket b \rrbracket = \text{false}}{\text{if } b \text{ then } p \text{ else } q \xrightarrow{\alpha} q'} \quad \text{RESTAU} \quad \frac{p \xrightarrow{\tau} p'}{(\nu a) p \xrightarrow{\tau} (\nu a) p'} \\
\\
\text{RESMU} \quad \frac{p \xrightarrow{\mu} p' \quad \text{chan}(\mu) \neq a}{(\nu a) p \xrightarrow{\mu} (\nu a) p'} \quad \text{COML} \quad \frac{p \xrightarrow{a!v} p' \quad q \xrightarrow{a?v} q'}{p|q \xrightarrow{\tau} p'|q'} \quad \text{COMR} \quad \frac{p \xrightarrow{a?v} p' \quad q \xrightarrow{a!v} q'}{p|q \xrightarrow{\tau} p'|q'} \quad \text{PARL} \quad \frac{p \xrightarrow{\alpha} p'}{p|q \xrightarrow{\alpha} p'|q} \\
\\
\text{PARR} \quad \frac{q \xrightarrow{\alpha} q'}{p|q \xrightarrow{\alpha} p|q'} \quad \text{SUML} \quad \frac{g_1 \xrightarrow{\alpha} g'_1}{g_1 + g_2 \xrightarrow{\alpha} g'_1} \quad \text{SUMR} \quad \frac{g_2 \xrightarrow{\alpha} g'_2}{g_1 + g_2 \xrightarrow{\alpha} g'_2}
\end{array}$$

Figure 2: LTS of VACCS

Note that we can derive the following rules for the derived forms $p \oplus q$ and Ω :

$$\frac{}{p \oplus q \xrightarrow{\tau} p} \quad \frac{}{p \oplus q \xrightarrow{\tau} q} \quad \frac{}{\Omega \xrightarrow{\tau} \Omega}$$

Example 2.1. Here is an example of a simple communication where an agent $p = a!v$ sends a value v along a channel a and in parallel another agent $q = a?(x).b!x$ waits to receive a value on a then sends the received value on a channel b :

$$p|q = a!v|a?(x).b!x \xrightarrow{\tau} \mathbf{0}|b!v \xrightarrow{b!v} \mathbf{0}|\mathbf{0}$$

Notice that unlike the receiver $a?(x).p$ on a , the sender $a!v$ on a does not have a continuation p . This is a particular trait of VACCS that highlights the asynchrony feature of VACCS: sending a value is non-blocking. It is non-blocking in the sense that sending something does not suspend computation. On the other hand receiving something does suspend computation: the process $a?(x).p$ will wait and suspend computation until it receive something on a , once it has received something it continues on by computing p so the prefix “ $a?(x)$ ” allows us to suspend p .

The action $a!v$ of sending is said to be *dual* to the action $a?v$ of receiving and vice-versa. The dual of a visible action μ is written $\bar{\mu}$.

When studying the λ -calculus, it is useful to have a notion of program equivalence to identify two programs that have similar behavior like for instance β -equivalence. When studying process algebras it is also useful to have a notion of equivalence so that we can identify processes that have similar behavior and the most basic way to say that two process p and q are equivalent is to say that they are *structurally congruent* (or just *congruent*) written $p \equiv q$. Some of the rules for congruence is given in Figure 3. Here in that figure, we are considering the rule CGSYM as part of the definition of $\equiv$ but in practice when doing Rocq formalization of process calculi, it is better to not have it in the definition of $\equiv$ and instead it is better to add symmetric version of each axioms of the congruence relation: for instance, not only we would have in the definition $p|\mathbf{0} \equiv p$ as axiom but also $p \equiv p|\mathbf{0}$ as axiom. And again, in practice when doing Rocq formalization of process calculi, it is actually better practice to isolate the rule of transitivity CGTRANS from the other rules to facilitate inversion on the congruence judgment by defining first say $\equiv_{\text{nt}}$ containing all the rule that are not the transitivity rule then defining $\equiv$ as the transitive closure of $\equiv_{\text{nt}}$.

CGREFL $\frac{}{p \equiv p}$	CGSYM $\frac{q \equiv p}{p \equiv q}$	CGTRANS $\frac{p \equiv r \quad r \equiv q}{p \equiv q}$	CGNU $\frac{p \equiv q}{(\nu a) p \equiv (\nu a) q}$	CGPARNIL $\frac{}{p \mathbf{0} \equiv p}$
CGPARASSOC $\frac{}{(p q) r \equiv p (q r)}$	CGPLUSASSOC $\frac{}{(g_1 + g_2) + g_3 \equiv g_1 + (g_2 + g_3)}$	CGPLUSCOMM $\frac{}{g_1 + g_2 \equiv g_2 + g_1}$	CGPARCOMM $\frac{}{p q \equiv q p}$	

Figure 3: Selected rules for structural congruence

2.3 Convergence

The notion of convergence is straightforward in the λ -calculus: any sequence of reductions must be finite. But in process calculi, we have two ways of rewriting a process: an internal move $\xrightarrow{\tau}$ or a visible move $\xrightarrow{\mu}$, so in this case, what is a “reduction”?

Naively, one might want say a reduction in this context correspond to any labelled transitions $\xrightarrow{\alpha}$. But here a reduction will actually mean an internal move $\xrightarrow{\tau}$. So our notion of convergence is the following: p converge if all sequences of τ -transitions are finite.

In order to reason on convergence rigorously and without too much pain, we define it as an inductive predicate without base case and with only one rule:

$$\frac{\text{CONV} \quad \forall p'. p \xrightarrow{\tau} p' \implies p' \downarrow}{p \downarrow}$$

It turns out that this definition actually has an equivalent formulation $p \downarrow_i \iff p \downarrow$ that has both a base case and inductive case like the usual inductive predicates we often see:

$$\frac{p \xrightarrow{\tau} _}{p \downarrow_i} \quad \frac{p \xrightarrow{\tau} _ \quad \forall p'. p \xrightarrow{\tau} p' \implies p' \downarrow_i}{p \downarrow_i}$$

The formulations of those inference rules are a bit strange but it is read like any other rules for inductively defined predicates: such predicates are the smallest ones such that the premise of the rules implies the conclusion of the rules. The inductive instance of $p \downarrow$ here is nested in an implication and universal quantifier as we can see in the premise of the rule CONV which is indeed unusual for defining a judgment. The reason for this strange nesting is that our goal is to avoid defining it on the syntax of the processes so that we can effortlessly apply our framework to any process algebra.

We will now extend the definition of convergence to *traces*. What we mean by a trace s is basically a list of visible actions:

$$\text{traces} \quad s ::= \varepsilon \mid \mu.s$$

We will of course write for simplicity just μ instead of $\mu.\varepsilon$ and concatenation of two traces s and s' will be written ss' . Defining the notion of convergence on trace requires us to define the notion of weak transition $p \xRightarrow{s} p'$.

The definition of weak transitions is given in Figure 4, and the definition of convergence on trace is given in Figure 5. Let us unpack those definitions to gain a better understanding of them.

$$\begin{array}{c}
\text{WTREFL} \\
\frac{}{p \xRightarrow{\varepsilon} p}
\end{array}
\qquad
\begin{array}{c}
\text{WTLATER} \\
\frac{p \xrightarrow{\tau} p_0 \quad p_0 \xRightarrow{s} p'}{p \xRightarrow{s} p'}
\end{array}
\qquad
\begin{array}{c}
\text{WTNOW} \\
\frac{p \xrightarrow{\mu} p_0 \quad p_0 \xRightarrow{s} p'}{p \xRightarrow{\mu.s} p'}
\end{array}$$

Figure 4: Weak transitions

$$\begin{array}{c}
\text{CNVNIL} \\
\frac{p \downarrow}{p \Downarrow \varepsilon}
\end{array}
\qquad
\begin{array}{c}
\text{CNVCONS} \\
\frac{p \downarrow \quad \forall p'. p \xRightarrow{\mu} p' \implies p' \Downarrow s}{p \Downarrow \mu.s}
\end{array}$$

Figure 5: Convergence on trace

First of all the definition of weak transition can actually be expressed in a more direct fashion as follows:

$$\begin{aligned}
p \xRightarrow{\varepsilon} p' &\iff p \xrightarrow{\tau}^* p' \\
p \xRightarrow{\mu} p' &\iff p \xrightarrow{\tau}^* _ \xrightarrow{\mu} _ \xrightarrow{\tau}^* p' \\
p \xRightarrow{\mu_1 \dots \mu_n} p' &\iff p \xRightarrow{\mu_1} _ \xRightarrow{\mu_2} \dots _ \xRightarrow{\mu_n} p'
\end{aligned}$$

i.e. $p \xRightarrow{\mu} p'$ means that p will activate the action μ at some point (modulo τ -transitions) and eventually reach p' after some τ -transitions; and $p \xRightarrow{\mu_1 \dots \mu_n} p'$ means that p will at some point activate μ_1 then at some point μ_2 and so on up to μ_n then it reaches p' by some τ -transitions. When going from p to p' with a weak transition, the rule WTLATER expresses the delay of the activation of an action μ or the delay of reaching p' ; and the rule WTNOW expresses the activation of an action μ and the rule WTREFL express that we reached our destination p' .

It is easier to reason by induction on the predicate especially in Rocq, for that reason we chose to reason with the inductive definition rather than the direct one. Although certain things might be easier with the direct definition but most of the time the inductive definition is nicer. We note some simple properties: (1) $p \xrightarrow{\tau} p'$ implies $p \xRightarrow{\varepsilon} p'$, (2) $p \xrightarrow{\mu} p'$ implies $p \xRightarrow{\mu} p'$.

The essential things to keep in mind for the predicate $p \Downarrow s$ is the proposition below:

Proposition 2.2. *The following holds:*

1. If $p \Downarrow s_1 s_2$ and $p \xRightarrow{s_1} p'$ then $p' \Downarrow s_2$.
2. If $p \Downarrow s$ then $p \Downarrow s'$ for any prefix s' of s .

We note also that $p \Downarrow \varepsilon$ just mean $p \downarrow$ and that the notion of convergence on trace generalizes in that manner the notion of convergence $p \downarrow$ that we first defined.

2.4 The must preorder

We define an inductive predicate MUST as in Figure 7 which make use of a decidable predicate GOOD defined in Figure 6. The definition might seems strange since we don't often see judgments with inductive instances nested in a bunch of logical connectives but it is read like any other inductive predicate just like we said for the notion of convergence: it's the smallest binary relation such that the premises implies the conclusion in all of the inference rules. This predicate $p \text{ MUST } e$ expresses liveness of p with respect to a process e that we usually refer to as a *test* or *experiment*. The inductive rule GOODLATER basically tells us that if

$\frac{\text{GOODSucc}}{\text{GOOD } \mathbf{1}}$	$\frac{\text{GOODPARL} \quad \text{GOOD } p}{\text{GOOD } (p q)}$	$\frac{\text{GOODPARR} \quad \text{GOOD } q}{\text{GOOD } (p q)}$	$\frac{\text{GOODPLUSL} \quad \text{GOOD } p}{\text{GOOD } (p + q)}$	$\frac{\text{GOODPLUSR} \quad \text{GOOD } q}{\text{GOOD } (p + q)}$
$\frac{\text{GOODNu} \quad \text{GOOD } p}{\text{GOOD } ((\nu a) p)}$	$\frac{\text{GOODIfTRUE} \quad \text{GOOD } p \quad \llbracket b \rrbracket = \text{true}}{\text{GOOD } (\text{if } b \text{ then } p \text{ else } q)}$	$\frac{\text{GOODIfFALSE} \quad \text{GOOD } q \quad \llbracket b \rrbracket = \text{false}}{\text{GOOD } (\text{if } b \text{ then } p \text{ else } q)}$		

Figure 6: GOOD predicate

1. the test e is not good at the moment.
2. $p|e$ does at least one τ -transition.
3. τ -transitions of the parallel composition $p|e$ propagates MUST.

Then p MUST e . Now, the inductive must-preorder $\sqsubseteq$ is then defined as

$$p \sqsubseteq q \iff \forall e. \ p \text{ MUST } e \text{ implies } q \text{ MUST } e$$

This preorder expresses preservation of liveness when going from process p to process q .

This preorder is known to have an alternative definition/characterization $\preceq_{\text{AS}}$ using the notion of *convergence on a trace* and *acceptance sets*. To give the definition of $\preceq_{\text{AS}}$ we first need to define two auxiliary definitions, namely the convergence condition and acceptance set condition respectively:

$$\begin{aligned}
p \preceq_{\text{cnv}} q &\stackrel{\text{def}}{\iff} \forall s. \ p \Downarrow s \text{ implies } q \Downarrow s \\
p \preceq_{\text{acc}} q &\stackrel{\text{def}}{\iff} \forall s. \ p \Downarrow s \text{ implies } \mathcal{A}(p, s) \ll \mathcal{A}(q, s)
\end{aligned}$$

Where the following definitions are used:

1. $\mathcal{C} \ll \mathcal{C}'$ if for any $C' \in \mathcal{C}'$, there exists $C \in \mathcal{C}$ with $C \subseteq C'$
2. $p \not\rightarrow_{\tau}$ means $p \not\rightarrow_{\tau} _$ (i.e. p cannot make a τ -transition)
3. $R(p)$ is called the *ready set* of p and it is defined to be the set of visible action that p can do: $R(p) = \{\mu \mid p \xrightarrow{\mu} _ \}$
4. $\mathcal{A}(p, s)$ is the set defined as follows:

$$\mathcal{A}(p, s) = \{R(p') \mid p \xRightarrow{s} p' \not\rightarrow_{\tau}\}$$

Then we write $p \preceq_{\text{AS}} q$ when when both $p \preceq_{\text{cnv}} q$ and $p \preceq_{\text{acc}} q$ holds. We note that the condition $\mathcal{A}(p, s) \ll \mathcal{A}(q, s)$ basically mean that any s -weak transitions of q can be *simulated* by p in the following sense: if q' is such that $q \xRightarrow{s} q' \not\rightarrow_{\tau}$ then there exists p' with $p \xRightarrow{s} p' \not\rightarrow_{\tau}$ and any visible actions doable by p' is also doable by q' .

3 Main contributions

We have been studying the compositionality properties of the must preorder through five constructs of VACCS: $\tau.p$, $a?(x).p$, $g_1 + g_2$, $(\nu a)p$, and $p|q$. For the sake of clarity, let us state precisely what

$$\begin{array}{c}
\text{GOODNOW} \\
\frac{\text{GOOD } e}{p \text{ MUST } e}
\end{array}
\quad
\begin{array}{c}
\text{GOODLATER} \\
\frac{
\begin{array}{l}
\neg \text{GOOD } e \quad \forall p'. p \xrightarrow{\tau} p' \implies p' \text{ MUST } e \\
p \parallel e \xrightarrow{\tau} _ \quad \forall e'. e \xrightarrow{\tau} e' \implies p \text{ MUST } e' \\
\forall p' \mu. p \xrightarrow{\mu} p' \wedge e \xrightarrow{\mu} e' \implies p' \text{ MUST } e'
\end{array}
}{p \text{ MUST } e}
\end{array}$$

Figure 7: MUST predicate

we mean by compositionality here: given a preorder $\leq$ what we mean by $\leq$ composes with those respective construct is respectively:

$$\begin{aligned}
p \leq q &\implies \tau.p \leq \tau.q \\
(\forall \sigma \text{ substitution } . p\sigma \leq q\sigma) &\implies a?(x).p \leq a?(x).q \\
g_1 \leq g'_1 \text{ and } g_2 \leq g'_2 &\implies g_1 + g_2 \leq g'_1 + g'_2 \\
p \leq q &\implies (\nu a)p \leq (\nu a)q \\
p_1 \leq p'_1 \text{ and } p_2 \leq p'_2 &\implies p_1 | p_2 \leq p'_1 | p'_2
\end{aligned}$$

On a side note, in our work we had to admit some lemmas on substitution with De Bruijn indices because of its technicalities and even though we have the tool `autosubst 2` that can help us with De Bruijn indices, the tool does not work so well for generating the syntax in Rocq for VACCS.

Now back to compositionality, the reason behind why we need that $p\sigma \leq q\sigma$ for all substitutions⁴ σ instead of just $p \leq q$ as hypothesis in the compositionality for receivers is that since a receiver $a?(x).p$ can receive any values sent along the channel a , we need to take into account all possible value that can be sent and for that reason, we need to require that $p\sigma \leq q\sigma$ for all σ . Now one might say that if we are working in a reasonable calculus, shouldn't substitutions respect the relations we are working with? And thus with that we could simplify the hypothesis into just $p \leq q$; but unfortunately in our Rocq code, like previously said, I had to leave things related to substitutions aside to avoid spending too much time on technicalities.

Let us also note that $+$ actually does not compose which the must preorder, it does not composes with the contextual characterization nor the alternative characterization. The failure of compositionality for $+$ can be seen with a simple counterexample that mixes $+$ with τ and is one of the reason why some people advocate against the use of the tau construct $\tau.p$ in process algebras, more on that matter is discussed in the paper [6].

Mixing $+$ with τ also illustrate one of the reason why agents in the syntax of VACCS are separated into two syntactic categories (processes and guards) and how τ 's ruin the benefit of guards: It would have been nice to have the property " $p + q \sqsubseteq p$ " but the latter is actually not true in general (👉) as we can observe by taking $p = \mathbf{0}$ and $q = \tau.a!v$. Specifically, allowing non-deterministic choice to send values is too powerful so guards filter out the possibility of writing " $a!v + _$ " but the tau construct let us counter this restriction by allowing us to write $(\tau.a!v) + _$ and since the internal action τ has invisible effects, $(\tau.a!v) + _$ almost behave as how " $a!v + _$ " would have behaved.

Although $+$ fails to compose with the must preorder in VACCS, a restricted version of it does compose, namely the internal sum construct $p \oplus q$.

⁴What we mean by substitution here are substitutions that replace value variables x with values v

3.1 The contextual characterization

In this subsection we will talk about the compositionality properties of the must preorder with respect to the contextual characterization $\sqsubseteq$.

All of our compositionality proofs here for $\sqsubseteq$ relies on intermediate lemmas about the MUST predicate except for some of the construct: the case for $\tau.p$ and $a?(x).p$ can both be proven directly by an induction without an intermediate lemma.

Proposition 3.1. *For any p and q , we have:*

$$\text{👉 } p \sqsubseteq q \implies \tau.p \sqsubseteq \tau.q$$

$$\text{👉 } (\forall \sigma. p\sigma \sqsubseteq q\sigma) \implies a?(x).p \sqsubseteq a?(x).q$$

To get a feel of what those induction look like, let us just describe here the case of the process $\tau.p$: what we want to show is that for any p and q , $p \sqsubseteq q$ implies $\tau.p \sqsubseteq \tau.q$. So fix p and q and assume that $p \sqsubseteq q$. Then to show $\tau.p \sqsubseteq \tau.q$ we unfold the definition of $\sqsubseteq$ which tells us that we have to show that given e with $\tau.p \text{ MUST } e$, we must show $\tau.q \text{ MUST } e$. So we proceed by an induction on the hypothesis $\tau.p \text{ MUST } e$ and that is it.

The proof of compositionality for the restriction operator ν and parallel composition $|$ are just an immediate application of some intermediate lemmas proven by induction on MUST that relates those operators to the must predicate MUST. Let us write $\text{no } q$ to mean that q does not contain the process **1**. Now, let us describe the relevant lemma for $|$.

Lemma 3.2. *For all p, q and e ,*

$$\text{👉 } p|q \text{ MUST } e \implies p \text{ MUST } q|e$$

$$\text{👉 } p \text{ MUST } q|e \text{ and } \text{no } q \implies p|q \text{ MUST } e$$

The idea behind this lemma is that the predicate $p \text{ MUST } e$ describe whether p will at some point make the test e successful by running $p|e$. So in particular running $(p|q)|e$ is the same as running $p|(q|e)$ because these two processes are structurally congruent. Though as we can see we don't exactly have an equivalence in the above lemma and have a side condition $\text{no } q$ for one of the implications and the reason for that is that in our definition of VACCS' syntax, we include experiments/tests e as part of the processes of VACCS, in particular we allow processes q to contain the success construct **1**. Its presence is what disallows us to get a nice equivalence statement. We decided to mix processes and experiments together for convenience but in theory, they can be separated out.

Because of the side condition $\text{no } q$ in Lemma 3.3, we also need such side condition in the statement of the compositionality of $|$. Now, to prove the compositionality for $|$, it suffices to prove the following lemma, then use it twice with the fact that parallel composition $|$ is commutative up to structural congruence.

Lemma 3.3 (👉). *For all p_1, p_2, q , $p_1 \sqsubseteq p_2$ and $\text{no } q$ implies $p_1|q \sqsubseteq p_2|q$.*

So the above lemma then gives the compositionality for $|$.

Proposition 3.4 (👉). *For all p_1, p'_1, p_2, p'_2 , if $p_1 \sqsubseteq p'_1$, $p_2 \sqsubseteq p'_2$, $\text{no } p'_1$ and $\text{no } p_2$ then $p_1|p_2 \sqsubseteq p'_1|p'_2$.*

The relevant intermediate lemma to prove the compositionality for ν basically state that under a side condition saying that e does not communicate on the restrained channel a , we have the equivalence between $(\nu a)p \text{ MUST } e$ and $p \text{ MUST } e$ and since we use De Bruijn indices in our Rocq code, the side condition is replaced by an appropriate increment of channel indices of e in our Rocq code. So from the intermediate lemma we get also the compositionality for ν .

Proposition 3.5 (👉). *For all p and q , if $p \sqsubseteq q$ then $(\nu a)p \sqsubseteq (\nu a)q$.*

The compositionality of $\oplus$ is proven in a similar fashion as for $|$ by proving the special case version where we fix one branch of $\oplus$.

Lemma 3.6 (👉). *For all p_1, p_2, q , $p_1 \sqsubseteq p_2$ implies $p_1 \oplus q \sqsubseteq p_2 \oplus q$.*

Then applying twice the special case compositionality property above with the help of structural congruence gives the full compositionality property. Though, in our Rocq code, we decided to leave the special case as it is and focus on other things; and the same goes for the compositionality for $\oplus$ with the alternative characterization: we decided to leave the special case as it is.

Now, as stated before, the general sum $+$ does not compose with $\sqsubseteq$. The following counterexample (👉, 👉) for compositionality here is inspired (and adapted) from the one in the paper [6]: take $p_1 = \mathbf{0}$, $p_2 = \tau.\mathbf{0}$ and $q = \tau.a!v$, then $p_1 \sqsubseteq p_2$ but $p_1 + q \not\sqsubseteq p_2 + q$. The reason for failure here is that the test $e = a?(_).\mathbf{1}$ gives us $p_1 + q \text{ MUST } e$ but fails at giving $p_2 + q \text{ MUST } e$, and precisely the parallel composition $(p_2 + q)|e$ does not propagate the must predicate: $(p_2 + q)|e \xrightarrow{\tau} \mathbf{0}|e$ but $\mathbf{0} \text{ MYST } e$ because e is not good at the moment and can't be good later since the process $\mathbf{0}|e$ is stuck (no τ -transition is possible).

3.2 The alternative characterization

To prove the compositionality for the following four constructs, a common strategy is used.

Proposition 3.7. *For any p, p_1, p_2 and q , we have:*

- 👉 $p \preceq_{\text{AS}} q \implies \tau.p \preceq_{\text{AS}} \tau.q$
- 👉 $(\forall \sigma. p\sigma \preceq_{\text{AS}} q\sigma) \implies a?(x).p \preceq_{\text{AS}} a?(x).q$
- 👉 $p \preceq_{\text{AS}} q \implies (\nu a) p \preceq_{\text{AS}} (\nu a) q$
- 👉 $p_1 \preceq_{\text{AS}} p_2 \implies p_1 \oplus q \preceq_{\text{AS}} p_2 \oplus q$.

Namely, the strategy consist in proving two things: proving that the given construct composes with $\preceq_{\text{cnv}}$ and proving that the given construct composes with $\preceq_{\text{acc}}$. To get a feel of how the proofs goes, a detailed pen and paper proof for the compositionality with $\preceq_{\text{AS}}$ for the tau and receiver constructs is given in the appendix.

The alternative characterization of the must preorder is given through the use of a bunch of auxiliary concepts: weak transition, convergence, convergence on a trace, acceptance sets. So reasoning on the preorder through its alternative characterization requires proving a bunch of auxiliary lemmas on those notions. The auxiliary lemmas that talk about weak transitions $p \xRightarrow{s} p'$, convergence $p \downarrow$ and convergence on trace $p \Downarrow s$ are usually proven by induction on the respective judgments; although the relevant auxiliary lemmas for proving the compositionality for $\oplus$ did not require to do inductions and actually an inversion on the judgments are sufficient: this is because of the way the semantics of $g_1 + g_2$ is defined.

Now, for the general sum $+$, we also have failure of compositionality like for the contextual characterization and in fact the same counterexample that we gave before still works. Recall the counterexample that we gave for composition of $\sqsubseteq$ with $+$: take $p_1 = \mathbf{0}$, $p_2 = \tau.\mathbf{0}$ and $q = \tau.a!v$. Then just like for $\sqsubseteq$, we also have $p_1 \preceq_{\text{AS}} p_2$ but $p_1 + q \not\preceq_{\text{AS}} p_2 + q$ (👉, 👉). Although $+$ behaves nicely with convergence like we want, it does not behave so well with the acceptance set condition; indeed, we have:

$$\begin{aligned} \mathcal{A}(p_1 + q, \varepsilon) &= \{R(a!v)\} = \{\{a!v\}\} \\ \mathcal{A}(p_2 + q, \varepsilon) &= \{R(\mathbf{0}), R(a!v)\} = \{\emptyset, \{a!v\}\} \end{aligned}$$

but no set in $\mathcal{A}(p_1 + q, \varepsilon)$ can be a subset of $\emptyset$, thus violating the acceptance set condition.

It might seem strange that we managed to obtain a counterexample for the composition of $\preceq_{AS}$ with $+$ since there is actually a proof of the compositionality of $+$ with the alternate characterization of the must preorder in Hennessy's book [4], namely the corollary 4.4.7 in the book gives a proof of compositionality for $+$. The reason why things go well for Hennessy but not us is just that we use slightly different definitions. Hennessy's ready sets are the sets $S(p) = \{\mu | p \xrightarrow{\mu} _ \}$ while our notion of ready sets are the sets $R(p) = \{\mu | p \xrightarrow{\mu} _ \}$ this is exactly the reason why his proof work in his book and the reason why his proof won't work with our definition.

Now, for the compositionality for $|$, things are complicated. Recall that up until now in order to prove compositionality for $\preceq_{AS}$ with respect to a construct we just had to prove that the given construct composes with $\preceq_{cnv}$ and $\preceq_{acc}$ separately; but this won't work for compositionality with $|$. Indeed, $\preceq_{cnv}$ does not compose with $|$: take $p_1 = \mathbf{0}$, $p_2 = a!v$ and $q = a?(_).\Omega$ then we have $p_1 \preceq_{cnv} p_2$ trivially because both processes converge on all traces, however $p_1|q \not\preceq_{cnv} p_2|q$ since $p_1|q$ converge on the empty trace but $p_2|q$ diverge on the empty trace.

We will discuss in the next subsection, some attempts at tackling the problem of proving the compositionality for $|$ and specifically the convergence part.

3.3 Convergence of parallel composition

Note that up until now, in order to prove the $\preceq_{cnv}$ part of compositionality, we basically had to rely on some lemma that relate the convergence of the given construct with the convergence of a subprocess of the given construct, for instance for $\tau.p$, we use a lemma that state that $\tau.p$ converge on s if and only if p converge on s . So obviously, when proving the convergence part for compositionality for $|$, we would need some lemma that relates the convergence of $p|q$ with the convergence of p and q and maybe exploit the hypotheses that we have.

It is easy to see by a simple induction (on $p|q \Downarrow s$) that on any traces, the convergence of $p|q$ implies the convergence of both p and q .

Lemma 3.8 (👉, 👈). $p|q \Downarrow s$ implies $p \Downarrow s$ and $q \Downarrow s$.

Though the reverse implication of the above lemma is false; one possible reason is that even if p and q converge on their own, $p|q$ might diverge by unblocking a suspended computation: take $p = a?(x).\Omega$ and $q = a!v$ then these processes converge on their own but

$$p|q = a?(_).\Omega | a!v \xrightarrow{\tau} \Omega | \mathbf{0} \xrightarrow{\tau} \Omega | \mathbf{0} \xrightarrow{\tau} \dots$$

Another possible reason for failure of convergence is that even if p and q converge on their own, $p|q$ might diverge because p and q interact infinitely many often thus giving an infinite sequence of computation: take $p = \text{rec}X.(a!v | b?(_).X)$ and $q = \text{rec}X.(b!v | a?(_).X)$:

$$\begin{aligned} p|q &= \text{rec}X.(a!v | b?(_).X) | \text{rec}X.(b!v | a?(_).X) \\ &\xrightarrow{\tau} \cdot \xrightarrow{\tau} a!v | b?(_).p | b!v | a?(_).q \\ &\xrightarrow{\tau} \mathbf{0} | b?(_).p | b!v | q \\ &\xrightarrow{\tau} \mathbf{0} | p | \mathbf{0} | q \equiv p|q \xrightarrow{\tau} \dots \end{aligned}$$

In both those examples we had the issue that the convergence of $p|q$ is hindered by the fact that p and q communicate with each other. So one possible question is what if we assume that they do not communicate with each other, will that assumption allows us to force the convergence of $p|q$?

The answer is positive for empty traces but negative for non-empty traces, in the latter case an extra assumption on traces is needed.

Let us first define formally what it means for processes to not communicate through the following relation.

$$\begin{array}{c}
\text{ZIPNIL} \\
\hline
\varepsilon \text{ zip}(\varepsilon, \varepsilon)
\end{array}
\quad
\begin{array}{c}
\text{ZIPCONSL} \\
\hline
\frac{s \text{ zip}(s_1, s_2)}{\mu.s \text{ zip}(\mu.s_1, s_2)}
\end{array}
\quad
\begin{array}{c}
\text{ZIPCONSR} \\
\hline
\frac{s \text{ zip}(s_1, s_2)}{\mu.s \text{ zip}(s_1, \mu.s_2)}
\end{array}
\quad
\begin{array}{c}
\text{ZIPDUAL} \\
\hline
\frac{s \text{ zip}(s_1, s_2)}{s \text{ zip}(\mu.s_1, \bar{\mu}.s_2)}
\end{array}$$

Figure 8: Zipping rules

Definition 1 (🔴). We write $\text{nocom } p \ q$ when for all $\mu_p, \mu_q, s_1, s_2, s'_1, s'_2, p \xRightarrow{s_1 \mu_p s_2} _$ and $q \xRightarrow{s'_1 \mu_q s'_2} _$ implies μ_p and μ_q are not dual.

We then have the following.

Lemma 3.9 (🔴). If $p \downarrow$ and $q \downarrow$ and $\text{nocom } p \ q$, then we have $p|q \downarrow$.

The proof of the above lemma is done by induction on $p \downarrow$ with an inner induction on $q \downarrow$. We could probably factorize some part of the proof out to avoid nesting inductions but the factoring might be unsightly. Another possible way to avoid the nested induction would be to express the predicate $\text{nocom } p \ q$ inductively and do induction on it but it is not clear how to express this predicate inductively.

Lemma 3.9 and Lemma 3.8 tell us that under the hypothesis $\text{nocom } p \ q$, the convergence $p|q \downarrow$ is equivalent to the conjunction of $p \downarrow$ and $q \downarrow$. In other words if p and q do not communicate then the convergence (on the trace ε) of the parallel composition is equivalent to their convergence (on the trace ε). Unfortunately, this does not generalize nicely to arbitrary traces s . Indeed, we can refute the general trace analogue of Lemma 3.9 by taking $p = a?(_).\Omega$ and $q = b?(_).\mathbf{0}$ then we have that both processes don't communicate and both converge on $s = b?v.a?v$, but

$$p|q = a?(_).\Omega \mid b?(_).\mathbf{0} \xRightarrow{b?v} a?(_).\Omega \mid \mathbf{0} \xRightarrow{a?v} \Omega \mid \mathbf{0} \not\Downarrow \varepsilon$$

In other words, in this example we have $p \downarrow s$ and $q \downarrow s$ and $\text{nocom } p \ q$ but $p|q \not\downarrow s$. One way to force the convergence of $p|q$ on the general trace s is to furthermore assume that not only p and q converge on s but also on all “unzipping” of s .

Lemma 3.10 (🔴). Assume $p \downarrow s_1$ and $q \downarrow s_2$ for every s_1 and s_2 such that $s \text{ zip}(s_1, s_2)$; and assume $\text{nocom } p \ q$. Then $p|q \downarrow s$.

Before defining and discussing the notion of (un)zipping in the next subsection, let us give some remarks and observations on the convergence of $p|q$. It might be tempting to say that $p|q \downarrow s$ would imply that $p \downarrow s_1$ and $q \downarrow s_2$ for any given s_1 and s_2 that zips into s . But this is false (regardless of whether p and q communicate or not), note that $\varepsilon \text{ zip}(a?v, a!v)$ and take $p = a?(_).\Omega$ and $q = \mathbf{0}$. Then $p|q \downarrow \varepsilon$ but $p \not\downarrow a?v$.

What this and everything up to this point in this subsection hint at us is that when ensuring the convergence of $p|q$, we are faced with two difficulties: (1) the possibility of communication between p and q , (2) the definition itself of convergence on a trace.

3.4 Zipping

Let us define and explain what are zippings and unzippings. The only way a parallel composition $p|q$ can do a visible action μ throughout its evolution is either through p or through q , i.e. either p is the one that at some point activate μ or q is the one that at some point activate μ . So we can track all sequence of visible actions that $p|q$ can do from the ones that p and q can do.

When tracking sequences of visible actions of $p|q$ from the ones of p and q , we will also have to take into account within our recordings of visible actions of p and q the ones that p and q can do but are not visible to $p|q$ (the communications between p and q).

The judgment $s \text{ zip}(s_1, s_2)$ whose definition is given in Figure 8 formalizes this idea of tracking the visible actions of $p|q$. The definition tells us that s a sequence of visible actions that $p|q$ can do is obtained from the visible actions s_1 of p and the visible actions s_2 of q . There are a few ways to read $s \text{ zip}(s_1, s_2)$: s is a zipping of s_1 and s_2 , s unzips into s_1 and s_2 , and a reverse way to read it is that s_1 and s_2 zips into s .

The rules of Figure 8 basically tells us that to construct unzippings of s , it suffices to start from two empty traces $s_1 = \varepsilon$ and $s_2 = \varepsilon$ then iterate through the elements μ of s one by one, and at each iteration we can chose to put μ in either s_1 or in s_2 . And at each iteration (even after we emptied s), before putting μ in one of the s_i , we can chose to add canceling dual actions as much as we want in s_1 and s_2 . In particular, from that algorithm, we can for instance see that a trace zips into itself in the following way: $s \text{ zip}(\varepsilon, s)$ and $s \text{ zip}(s, \varepsilon)$ for any s .

The definition we give here for this inductive predicate follows from the definition given in Hennessy's book [4] (in section 5.3). In the book's definition of the predicate, the cons are done on the right " $s.\mu$ " whereas in our definition the cons are done on the left " $\mu.s$ " following traditional ML notations. This difference is just some aesthetic difference and does not matter at all, however when relating zip with prefixes of traces it is better (otherwise we would have to reverse the traces in the statements we want to prove) to use a definition closer to the one in Hennessy's book and to view traces as lists where the cons happens on the right. The definition that we use in our Rocq formalization uses the ML style list definition of zip as presented in Figure 8 which goes along pretty well with the definitions of weak transitions and convergence on trace that we use in our formalization.

The notion of zipping allows us to completely characterize weak transitions of parallel compositions $p|q \xRightarrow{s} P$ with the following fundamental lemmas of zipping.

Lemma 3.11 (Unzipping, 🍷). *If $p|q \xRightarrow{s} p'|q'$, then there exists s_1 and s_2 that satisfy $s \text{ zip}(s_1, s_2)$ and satisfy: $p \xRightarrow{s_1} p'$ and $q \xRightarrow{s_2} q'$.*

Lemma 3.12 (Zipping, 🍷). *If $s \text{ zip}(s_1, s_2)$ and $p \xRightarrow{s_1} p'$ and $q \xRightarrow{s_2} q'$, then $p|q \xRightarrow{s} p'|q'$.*

The unzipping lemma is proven by induction on $p|q \xRightarrow{s} p'|q'$; and the zipping lemma is proven by induction on the judgment $s \text{ zip}(s_1, s_2)$. The combination of these lemmas tells us that $p|q \xRightarrow{s} p'|q'$ if and only if $p \xRightarrow{s_1} p'$ and $q \xRightarrow{s_2} q'$ for some s_1 and s_2 satisfying $s \text{ zip}(s_1, s_2)$. This indeed characterize all weak transitions of $p|q$ by the following inversion lemma.

Lemma 3.13 (🍷). *If $p|q \xRightarrow{s} P$ then $P = p'|q'$ for some p' and q' .*

Here are some simple examples of zippings, we have for instance

$$\varepsilon \text{ zip}(a?v, a!v), \quad a?v_1.b!v_2 \text{ zip}(a?v_1, b!v_2).$$

4 Discussion and future works

At the end of Section 3.2, we saw that proving compositionality for $|$ with the alternate characterization by proving compositionality with $\preceq_{\text{cnv}}$ and $\preceq_{\text{acc}}$ separately won't work because $\preceq_{\text{cnv}}$ fails to compose with $|$, and in fact in order to prove the convergence condition for the compositionality with $|$ i.e. the convergence of $p_2|q$ knowing $p_1|q \Downarrow s$ and $p_1 \preceq_{\text{AS}} p_2$, we would need to not only exploit the convergence condition of the hypothesis $p_1 \preceq_{\text{AS}} p_2$ but also its acceptance sets condition i.e. convergence alone won't do the trick this time unlike for the $\preceq_{\text{cnv}}$ proof for the other constructs. And moreover, in section 3.3, we saw that even though the convergence of $p|q$ forces the convergence of p and q , the converse is not always true and requires some very strong additional hypothesis to ensure it.

We would like of course to not have to rely on such strong hypotheses so we have to somehow make use of acceptance sets when proving the convergence part for the compositionality with $|$ and this is basically what Hennessy does in his book. Hennessy does the compositionality proof

for $|$ by contradiction by using the acceptance set condition (which allows to simulate a weak transition) to construct an infinite derivation that would yield a contradiction. The issue with his proof is that it rely on a lot of handwavy and implicit arguments.

One possible thing to do in the future is thus to study in detail his proof and probably define more concepts that would allows us to make his proof rigorous (to the point of being able to formalize it in Rocq). For instance in his proof, he makes use of the fact that if two processes p and q converge on their own and their parallel composition $p|q$ diverge then p and q “interact infinitely many often”; our current definition of divergence is just plainly the negation of convergence but this alone does not seem to give sufficient data to prove this lemma so we would need to have a more “flexible” characterization of divergence; and also define formally what “interact infinitely many often” for two processes p and q means. One attempt at defining this latter property would be to say that for all s_p and s_q , if $p \xrightarrow{s_p} _$ and $q \xrightarrow{s_q} _$ then there exists μ , s'_p and s'_q such that $p \xrightarrow{s_p s'_p \mu} _$ and $q \xrightarrow{s_q s'_q \bar{\mu}} _$. However it is unclear weather this definition would be flexible enough.

Once we managed to obtain a Rocq formalization of Hennessy’s proof, one thing we would like to do is to tweak the various definitions so that we get a constructive proof.

Finally, we would like to study the compositionality of the must preorder through its coinductive characterization in Rocq just like we did for the other characterizations.

Once we have done all of that, we would like to establish a benchmark between the number of lines of code needed for the compositionality proofs for the three characterization of the must preorder in order to determine which one is “easier”.

Informally speaking, it seems (to me) that the contextual characterization is the easiest to use since it does not make uses of auxiliary definitions like convergence and weak transition and thus does not seem to require too much auxiliary lemmas.

Some people in the past seems to argue that the alternative characterization is easier to use than the contextual characterization. And the author of this report suspects that what those people meant is that the alternative characterization is easier to use for constructing and providing concrete example of p and q with p less than q rather than to be used for proving general properties.

We saw that proving general properties with the contextual characterization is usually just some straightforward inductions, whereas the alternative characterization usually requires a lot of auxiliary lemmas.

Note also that in terms of constructing concrete p and q , it is easier to construct such processes as counterexamples rather than examples for the preoder. We can notice that for instance in Hennessy’s book, after Hennessy introduces the preorder as well as his first process algebra in the book, the first examples for the contextual characterization that he gives are actually non-examples rather than examples (Examples 2.2.3 of his book). We can also notice it in our Rocq development (👉) where we have a significant amount of counterexamples of things related to the preorder.

5 Conclusion

We studied the compositionality properties of the must preoder through its contextual and alternative characterization by studying the cases of five of the constructs of VACCS: tau, receiver, sum, restriction, and parallel composition. Our study of compositionality has been formalized with the Rocq proof assistant.

We saw that the case of the parallel composition for the compositionality with respect to the alternative characterization is a tedious problem. And we saw that the difficulty of ensuring the convergence of the parallel composition $p|q$ for the compositionality property that we wish to prove lies in the possility that p and q can communicate and the definition of convergence on trace. We saw that in order to get without acceptance sets the equivalence between the convergence of $p|q$ and the subprocesses p and q , we likely need very strong side conditions as hypotheses.

Unfortunately at the moment, we couldn't completely achieve our original goals for the internship as we didn't have the opportunity to work on the coinductive characterization of the preorder and realized that the compositionality for $\downarrow$ with respect to $\preceq_{AS}$ is actually a tedious task and spent a lot of time on it.

Bibliography

- [1] Giovanni Bernardi, Iliara Castellani, Paul Laforgue, and Léo Stefanescu. Constructive characterisations of the must-preorder for asynchrony. In Viktor Vafeiadis, editor, *Programming Languages and Systems - 34th European Symposium on Programming, ESOP 2025, Held as Part of the International Joint Conferences on Theory and Practice of Software, ETAPS 2025, Hamilton, ON, Canada, May 3-8, 2025, Proceedings, Part I*, volume 15694 of *Lecture Notes in Computer Science*, pages 88–116. Springer, 2025.
- [2] Giovanni Bernardi, Hugo Férée, and Gaëtan Lopez. A uniform characterisation of the (a)synchronous must-preorder. *CoRR*, abs/2605.02362, 2026.
- [3] Iliara Castellani and Matthew Hennessy. Testing theories for asynchronous languages. In Vikraman Arvind and Ramaswamy Ramanujam, editors, *Foundations of Software Technology and Theoretical Computer Science, 18th Conference, Chennai, India, December 17-19, 1998, Proceedings*, volume 1530 of *Lecture Notes in Computer Science*, pages 90–101. Springer, 1998.
- [4] Matthew Hennessy. *Algebraic theory of processes*. MIT Press series in the foundations of computing. MIT Press, 1988.
- [5] Savan Kan and Sergueï Lenglet. Harmony in rocq. In Laura Bocchi and Burcu Kulahcioglu Ozkan, editors, *Formal Techniques for Distributed Objects, Components, and Systems - 46th IFIP WG 6.1 International Conference, FORTE 2026, Held as Part of the 21st International Federated Conference on Distributed Computing Techniques, DisCoTec 2026, Urbino, Italy, June 8-12, 2026, Proceedings*, volume 16589 of *Lecture Notes in Computer Science*, pages 20–37. Springer, 2026.
- [6] Rocco De Nicola and Matthew Hennessy. CCS without tau's. In Hartmut Ehrig, Robert A. Kowalski, Giorgio Levi, and Ugo Montanari, editors, *TAPSOFT'87: Proceedings of the International Joint Conference on Theory and Practice of Software Development, Pisa, Italy, March 23-27, 1987, Volume 1: Advanced Seminar on Foundations of Innovative Software Development I and Colloquium on Trees in Algebra and Programming (CAAP'87)*, volume 249 of *Lecture Notes in Computer Science*, pages 138–152. Springer, 1987.

6 Appendix: selected compositionality proofs on pen and paper

Let us now focus on the compositionality of $\tau.p$ with respect to $\preceq_{AS}$.

Lemma 6.1. *The following holds for any p and s :*

- 1. $p \downarrow \iff \tau.p \downarrow$
- 2. $p \Downarrow s \iff \tau.p \Downarrow s$

Proof. For 1, if $p \downarrow$ then $\tau.p \downarrow$ since the only τ -transition of $\tau.p$ is p ; the reverse implication is trivial. For 2, the implication “ $\Leftarrow$ ” comes from the fact that $-\Downarrow s$ preserves τ -transitions; and the implication “ $\Rightarrow$ ” is proven by induction on $p \Downarrow s$ using point 1. $\square$

Proposition 6.2. *For all p and q , $p \preceq_{AS} q$ implies $\tau.p \preceq_{AS} \tau.q$*

Proof. In two steps:

- We first show $\preceq_{\text{cnv}}$ composes with $\tau.p$: $p \preceq_{\text{cnv}} q \implies \tau.p \preceq_{\text{cnv}} \tau.q$; this is immediate from Lemma 6.1.
- We now show that $\preceq_{\text{acc}}$ composes with $\tau.p$:

$$p \preceq_{\text{acc}} q \implies \tau.p \preceq_{\text{acc}} \tau.q$$

Assume $p \preceq_{\text{acc}} q$ and let s be such that $\tau.p \Downarrow s$, we have from the latter hypothesis and with the help of Lemma 6.1 that $p \Downarrow s$ so the hypothesis $p \preceq_{\text{acc}} q$ gives $\mathcal{A}(p, s) \ll \mathcal{A}(q, s)$.

Our goal is to show $\mathcal{A}(\tau.p, s) \ll \mathcal{A}(\tau.q, s)$, so let $R(Q) \in \mathcal{A}(\tau.q, s)$, Q is such that the following holds:

$$\tau.q \xRightarrow{s} Q \not\rightarrow$$

so by inversion on the weak transition:

$$\frac{\tau.q \xrightarrow{\tau} q \quad q \xRightarrow{s} Q}{\tau.q \xRightarrow{s} Q}$$

we thus have:

$$q \xRightarrow{s} Q \not\rightarrow$$

And since $\mathcal{A}(p, s) \ll \mathcal{A}(q, s)$, there exists $R(P) \in \mathcal{A}(p, s)$ such that $R(P) \subseteq R(Q)$. By definition of acceptance sets, P satisfy:

$$p \xRightarrow{s} P \not\rightarrow$$

which allows us to build the following derivation:

$$\frac{\tau.p \xrightarrow{\tau} p \quad p \xRightarrow{s} P}{\tau.p \xRightarrow{s} P \not\rightarrow}$$

Thus giving $R(P) \in \mathcal{A}(\tau.p, s)$. So $R(Q)$ contains such $R(P)$ which implies that $\mathcal{A}(\tau.p, s) \ll \mathcal{A}(\tau.q, s)$ as desired. □

Proposition 6.3. *If $p\sigma \preceq_{\text{AS}} q\sigma$ for all substitution σ then $a?(x).p \preceq_{\text{AS}} a?(x).q$*

Proof. In two steps:

- We first show that $\preceq_{\text{cnv}}$ composes with $a?(x).p$:

$$(\forall \sigma. p\sigma \preceq_{\text{cnv}} q\sigma) \implies a?(x).p \preceq_{\text{cnv}} a?(x).q$$

Suppose the following holds:

$$\forall \sigma. p\sigma \preceq_{\text{cnv}} q\sigma \tag{*}$$

Then consider s with $a?(x).p \Downarrow s$. If $s = \varepsilon$ then we have to show $a?(x).q \Downarrow \varepsilon$ i.e. $a?(x).q \downarrow$ which is obviously true. Otherwise s has the form $s = \mu.\tilde{s}$. We want to show $a?(x).q \Downarrow \mu.\tilde{s}$ and to do so it suffices to show that it converge (trivial) and that it satisfies:

$$\forall Q. a?(x).q \xRightarrow{\mu} Q \implies Q \Downarrow \tilde{s}$$

So let Q be such that $a?(x).q \xRightarrow{\mu} Q$. By inversion, we must have $\mu = a?v$ for some v and:

$$\frac{a?(x).q \xrightarrow{\mu} q[v/x] \quad q[v/x] \xRightarrow{\varepsilon} Q}{a?(x).q \xRightarrow{\mu} Q}$$

By the semantics of receivers, we have $a?(x).p \xrightarrow{\mu} p[v/x]$ and by the hypothesis $a?(x).p \Downarrow \mu.\tilde{s}$, we have $p[v/x] \Downarrow \tilde{s}$ and using (*), we get $q[v/x] \Downarrow \tilde{s}$, and since we also have $q[v/x] \xRightarrow{\varepsilon} Q$, we thus obtain $Q \Downarrow \tilde{s}$.

- We now show that $\preccurlyeq_{\text{acc}}$ composes with $a?(x).p$:

$$(\forall \sigma. p\sigma \preccurlyeq_{\text{acc}} q\sigma) \implies a?(x).p \preccurlyeq_{\text{acc}} a?(x).q$$

So suppose $p\sigma \preccurlyeq_{\text{acc}} q\sigma$ for all σ . Then assume $a?(x).p \Downarrow s$. Our goal is to show:

$$\mathcal{A}(a?(x).p, s) \ll \mathcal{A}(a?(x).q, s)$$

Let $R(Q) \in \mathcal{A}(a?(x).q, s)$, then Q satisfy:

$$a?(x).q \xRightarrow{s} Q \not\rightarrow \quad (**)$$

We now proceed by case analysis.

- If $s = \varepsilon$: then by inversion on the weak transition (**), we obtain $Q = a?(x).q$ and thus:

$$R(Q) = \{a?v \mid v \text{ valeur}\} = R(a?(x).p) \in \mathcal{A}(a?(x).p, \varepsilon)$$

- If $s = \mu.\tilde{s}$: then by inversion on the weak transition (**) we see that $\mu = a?v$ for some v and that:

$$\frac{\overline{a?(x).q \xrightarrow{\mu} q[v/x]} \quad q[v/x] \xRightarrow{\tilde{s}} Q}{a?(x).q \xRightarrow{\mu.\tilde{s}} Q \quad (\not\rightarrow)}$$

Since we have by assumption that $a?(x).p \Downarrow \mu.\tilde{s}$ and $a?(x).p \xrightarrow{\mu} p[v/x]$ by the semantics of receivers, we thus get $p[v/x] \Downarrow \tilde{s}$ and since $p\sigma \preccurlyeq_{\text{acc}} q\sigma$ for all σ substitutions, we have in particular: $\mathcal{A}(p[v/x], \tilde{s}) \ll \mathcal{A}(q[v/x], \tilde{s})$. Combining the latter with the fact that we have $q[v/x] \xRightarrow{\tilde{s}} Q \quad (\not\rightarrow)$, gives us the existence of $R(P) \in \mathcal{A}(p[v/x], \tilde{s})$ with $R(P) \subseteq R(Q)$. The definition of P tells us that $p[v/x] \xRightarrow{\tilde{s}} P \not\rightarrow$ and this allows us to construct the following derivation:

$$\frac{\overline{a?(x).p \xrightarrow{\mu} p[v/x]} \quad p[v/x] \xRightarrow{\tilde{s}} P}{a?(x).p \xRightarrow{\mu.\tilde{s}} P \quad (\not\rightarrow)}$$

Which means that $R(p) \in \mathcal{A}(a?(x).p, s)$; thus $R(Q)$ contains such an $R(P)$ which shows that

$$\mathcal{A}(a?(x).p, s) \ll \mathcal{A}(a?(x).q, s)$$

as desired.

Now suppose $p\sigma \preccurlyeq_{\text{AS}} q\sigma$ for all σ . We then have that the premises of the two proven implications are true and thus we have $a?(x).p \preccurlyeq_{\text{cnv}} a?(x).q$ and $a?(x).p \preccurlyeq_{\text{acc}} a?(x).q$ which concludes the proof. $\square$